

APIS Zadanie 3 - Production Ready Chatbot

APIS – Zadanie 3 – Production Ready Chatbot

Sprawozdanie

Wstęp

1. Architektura systemu
 2. Model językowy i sposób integracji
 3. Prompt systemowy
 4. Historia rozmowy
 5. Parametry modelu
 6. Obsługa kontekstu (ocena 4.0)
 7. Obsługa błędów (ocena 4.5)
 8. Wdrożenie (ocena 5.0)
- Podsumowanie wymagań

APIS – Zadanie 3 – Production Ready Chatbot

Sprawozdanie

Autor: *Imię Nazwisko* **Przedmiot:** Aktualne Problemy Informatyki
Stosowanej – Geoinformatyka II st. **Data:** 2026

Wstęp

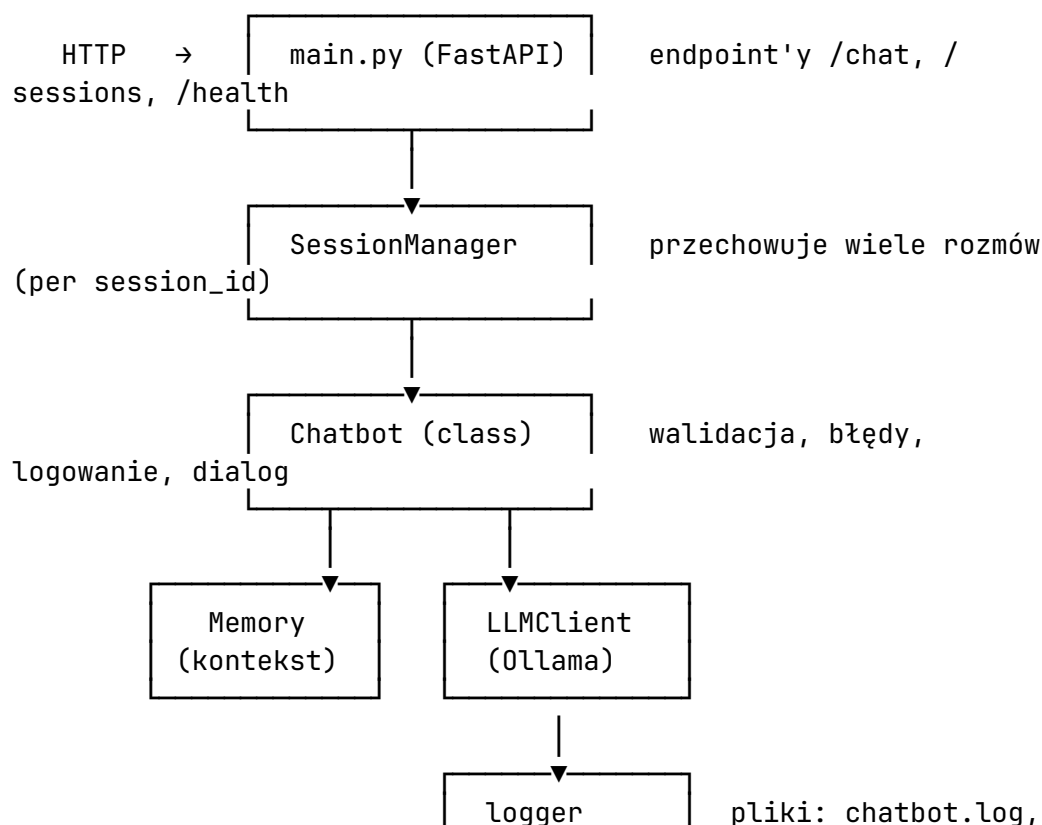
Celem projektu było zaprojektowanie i zaimplementowanie chatbota opartego na modelu językowym (LLM), spełniającego wymagania na ocenę 5.0 wg dokumentacji zadania. Rozwiązanie składa się z:

- klasy Chatbot integrującej model LLM, historię rozmowy i obsługę błędów,
- modułów wspierających (klient LLM, pamięć z kontrolą tokenów, logger, menedżer sesji),
- REST API zbudowanego w oparciu o FastAPI z obsługą wielu jednoczesnych sesji,
- dwóch notebooków demonstracyjnych (klasy Chatbot oraz REST API).

Cały kod znajduje się w katalogu `app/`, notebooki w `notebooks/`, logi w `logs/`.

1. Architektura systemu

System został zbudowany w warstwach o jasno rozdzielonych odpowiedzialnościach (separation of concerns). Dzięki temu każdy moduł można testować, podmieniać i rozwijać niezależnie.



errors.log



Komponenty:

Plik	Odpowiedzialność
app/config.py	Ustawienia (model, parametry, limity) wczytywane z .env
app/llm_client.py	Komunikacja z modelem LLM przez API kompatybilne z OpenAI
app/memory.py	Historia rozmowy + kontrola długości kontekstu (tokens)
app/chatbot.py	Logika dialogu: prompt + historia + obsługa błędów + logowanie
app/session_manager.py	Wiele równoległych sesji użytkowników (thread-safe)
app/schemas.py	Schematy Pydantic do walidacji i serializacji w API
app/main.py	Aplikacja FastAPI (REST endpoints)
app/logger_config.py	Konfiguracja loggera (plik + konsola)

Przepływ danych dla pojedynczego zapytania (REST):

1. Klient HTTP wysyła POST /sessions/{id}/chat z JSON-em {"message": "..."}.
2. FastAPI waliduje wejście przez schemat Pydantic (ChatRequest).
3. SessionManager.get(session_id) zwraca instancję Chatbot dla danej sesji, aktualizuje znacznik czasu ostatniego użycia.
4. Chatbot.chat(...) dodaje wiadomość do historii, prosi ConversationMemory o przygotowanie listy wiadomości w limicie tokenów.
5. LLMClient.generate(...) wysyła zapytanie do modelu (Ollama / OpenAI).
6. Odpowiedź zapisywana do historii, logowana, zwracana jako ChatResponse.
7. W razie błędu (timeout, brak połączenia, limit) – odpowiedni komunikat i wpis w logs/errors.log.

2. Model językowy i sposób integracji

Wybrany model: Llama 3.2 (3B parametrów) uruchamiany lokalnie przez narzędzie **Ollama**.

Dlaczego ten wybór: - **Brak limitów API** – Gemini-2.5-flash w darmowej wersji ma niskie quote'y zapytań na minutę, co utrudnia rozwój i testowanie. - **Prywatność** – wszystkie zapytania zostają lokalnie na maszynie. - **Koszt** – zero kosztów per zapytanie. - **Wystarczająca jakość** – Llama 3.2 3B świetnie radzi sobie z konwersacją w języku polskim i angielskim, dla potrzeb akademickiego chatbota jest w pełni wystarczająca. W razie potrzeby można uruchomić większą wersję (np. llama3.1:8b) jednym poleceniem `ollama pull`. - **Łatwa zamiana modelu** – Ollama wystawia interfejs zgodny z OpenAI, więc podmiana na model w chmurze (OpenAI, Anthropic, Groq) sprowadza się do zmiany jednej zmiennej w `.env`.

Sposób użycia (lokalnie):

```
ollama serve          # uruchamia serwer pod 0.0.0.0:11434
ollama pull llama3.2  # pobiera model
```

Biblioteki i uzasadnienie:

Biblioteka	Po co
openai	Klient HTTP zgodny z protokołem OpenAI Chat Completions – działa zarówno z OpenAI jak i Ollamą. Daje gotowy interfejs i typowane wyjątki.
pydantic + pydantic-settings	Walidacja danych wejściowych i wyjściowych, ładowanie konfiguracji z <code>.env</code> , automatyczna dokumentacja w FastAPI.
fastapi + uvicorn	Nowoczesny framework REST (asynchroniczny, szybki, z autogenerowaną dokumentacją Swagger / OpenAPI) oraz serwer ASGI.
tiktoken	Tokenizer kompatybilny z OpenAI, używany do estymacji

Biblioteka	Po co
	długości wiadomości przed wysłaniem do modelu.
python-dotenv	Wczytywanie zmiennych środowiskowych z pliku .env.

3. Prompt systemowy

Rola promptu systemowego: Definiuje “osobowość” i zachowanie bota. Jest wstrzykiwany jako pierwsza wiadomość każdej rozmowy (rola system). Model traktuje go z najwyższym priorytetem – określa styl, ton, język, granice wiedzy i sposób radzenia sobie z brakiem informacji.

Użyty prompt (z `app/config.py`):

```
Jesteś pomocnym, rzeczowym asystentem AI. Odpowiadasz krótko, konkretnie i zgodnie z prawdą. Jeżeli nie znasz odpowiedzi – przyznaj się, zamiast zmyślać. Domyślnie odpowiadasz po polsku, chyba że użytkownik pisze w innym języku.
```

Decyzje projektowe: - “**zgodnie z prawdą ... przyznaj się, zamiast zmyślać**” – obniża skłonność modelu do halucynacji, co jest jedną z głównych słabości LLM-ów. - “**Krótko, konkretnie**” – ogranicza długość odpowiedzi, oszczędza tokeny. - “**Domyślnie po polsku**” – Llama 3.2 ma tendencję do mieszania języków; jawna instrukcja stabilizuje wynik.

Prompt można podmienić indywidualnie dla sesji (parametr `system_prompt` w konstruktorze `Chatbot`), bez modyfikacji kodu.

4. Historia rozmowy

Struktura zapisywania: Historia jest listą słowników o formacie zgodnym z OpenAI Chat Completions:

```
[  
    {"role": "user", "content": "Cześć, kim jesteś?"},  
    {"role": "assistant", "content": "Jestem asystentem AI..."},  
    {"role": "user", "content": "Powiedz mi coś o LLM-ach."},  
    ...  
]
```

```
        {"role": "assistant", "content": "..."},  
    ]
```

System prompt **nie jest** częścią historii – dokleja się go dopiero przy budowaniu zapytania do modelu (zawsze jako pierwsza wiadomość). Pozwala to oddzielić “konfigurację” od “treści rozmowy”.

Sposób wykorzystania przy generowaniu odpowiedzi:

1. Użytkownik wysyła nową wiadomość → dopisujemy do history.
2. `ConversationMemory.build_messages()`:
 - sklejamy `[system_prompt]` + ostatnie_N_wiadomości_z_historii,
 - przycinamy okno tak, by suma tokenów nie przekroczyła `max_context_tokens`,
 - zwracamy gotową listę do wysłania.
3. Model dostaje **cały kontekst** (system prompt + przyciętą historię + nową wiadomość użytkownika) i odpowiada uwzględniając poprzednie tury.
4. Odpowiedź zapisujemy do historii (rola `assistant`).

Dzięki temu bot “**pamięta**” o czym była rozmowa – każda kolejna odpowiedź może odwoływać się do poprzednich.

5. Parametry modelu

W kliencie ustawiamy trzy główne parametry generacji (`app/config.py`, `app/llm_client.py`):

Parametr	Wartość	Wpływ na odpowiedź
temperature	0.7	Steruje “kreatywnością”. 0 – deterministyczna, monotonna; 2 – chaotyczna i z halucynacjami. 0.7 to dobry kompromis – odpowiedzi są naturalne, ale jeszcze rzeczowe.
top_p	0.9	Nucleus sampling – z rozkładu prawdopodobieństw

Parametr	Wartość	Wpływ na odpowiedź
max_tokens	1024	tokenów bierzemy tylko najbardziej prawdopodobne sumujące się do 0.9. Odsiewa “egzotyczne” tokeny, stabilizując odpowiedź bez wpływu na różnorodność typowych słów. Górny limit długości odpowiedzi. Chroni przed niekończącymi się odpowiedziami, kontroluje koszt i czas generowania. 1024 to ~750 słów – wystarczy na średnio złożoną odpowiedź.

Dlaczego nie domyślne wartości: - Domyślnie temperature=1.0 jest dla typowego chatbota zbyt kreatywne (zwiększa szansę halucynacji w odpowiedziach na pytania faktograficzne). - Bez top_p (lub przy top_p=1.0) model bierze pod uwagę cały rozkład, co przy wyższej temperaturze potęguje “dziwne” odpowiedzi. - Bez max_tokens model może generować bardzo długie odpowiedzi, co zwiększa latencję i koszt – limit 1024 jest pragmatyczny.

Wszystkie wartości można nadpisać przez .env bez zmiany kodu, co umożliwia eksperymenty bez rekompilacji.

Uwaga: Top-k nie jest jawnie ustawiane – Ollama używa domyślnej wartości 40, co jest standardowo dobrym wyborem. W razie potrzeby można je dodać.

6. Obsługa kontekstu (ocena 4.0)

Problem: Modele LLM mają ograniczone okno kontekstu (np. 8k tokenów). Jeśli wyślemy więcej, otrzymamy błąd BadRequest lub model “zapomni” początek rozmowy. Naiwne przycinanie “ostatnie N wiadomości” nie wystarczy, bo jedna wiadomość może mieć kilka tokenów albo kilka tysięcy.

Rozwiązanie: Algorytm sliding window oparty na tokenach (memory.py):

1. Liczymy tokeny w każdej wiadomości używając tiktoken (kodowanie `cl100k_base`).
2. System prompt zawsze włączony jako pierwszy (zachowuje “tożsamość” bota).
3. Idziemy od najnowszej wiadomości historii do tyłu, dodajemy póki suma tokenów $\leq \text{max_context_tokens}$ (domyślnie 3000).
4. Następnie odwracamy kolejność, by zachować chronologię.

PEŁNA HISTORIA (np. 30 wiadomości, 5000 tokenów):

m1 m2 m3 ... m20 m21 m22 m23 m24 m25 m26 m27 m28 m29 m30
 ← okno →
 (sliding window)

DO MODELU IDZIE:

[system_prompt, m24, m25, ..., m30] (suma ≤ 3000 tok)

Estymacja vs. dokładne liczenie: Ollama / Llama używają innego tokenizera niż OpenAI, ale dla limitowania kontekstu liczba tokenów z `cl100k_base` jest w tym samym rzędzie wielkości (błąd ~5-20%) i wystarcza w praktyce. Dokładny tokenizer wymagałby załadowania modelu Llama, co byłoby przesadą w warstwie pamięci.

Co się dzieje po przekroczeniu limitu: - Jeśli sama nowa wiadomość + system prompt nie mieszczą się w oknie – rzucający jest własny wyjątek `ContextOverflowError`, łapany przez Chatbot. Użytkownik dostaje czytelny komunikat “skrót zapytanie albo zresetuj rozmowę”, zdarzenie trafia do `errors.log`. Wiadomość użytkownika jest cofana, by nie zaśmiecać historii. - W normalnym przypadku stare wiadomości są po prostu **pomijane** w zapytaniu (ale **pozostają w `bot.history`** – można je np. wyświetlić użytkownikowi w UI).

Dodatkowo: `max_history_messages` (domyślnie 20) działa jako “twardy” limit liczby wiadomości – chroni przed bardzo długą historią, w której pojedyncze wiadomości są małe (np. “ok”, “tak”).

7. Obsługa błędów (ocena 4.5)

Możliwe błędy w systemie:

Błąd	Skąd	Jak obsługiwany
Serwer LLM nieosiągalny	Ollama nie działa, brak sieci	APIConnectionError → log ERROR, komunikat dla użytkownika, cofnięcie wiadomości
Timeout zapytania	Model za wolny, sieć	APITimeoutError → analogicznie
Zły klucz API	Błędna konfiguracja	AuthenticationError → log + komunikat
Limit zapytań przekroczony	OpenAI / Gemini quota	RateLimitError → log + komunikat “poczekaj chwilę”
Złe zapytanie (np. za długie)	Przekroczenie kontekstu po stronie modelu	BadRequestError → log + komunikat
Inne błędy HTTP	Serwer modelu	APIStatusError → log z kodem statusu
Pusta odpowiedź modelu	Bug modelu, zły prompt	ValueError → log + komunikat
Przekroczenie kontekstu	Za długa wiadomość	ContextOverflowError (własny) → log + prośba o skrócenie
Puste zapytanie	Użytkownik	Walidacja w Chatbot.chat + walidacja Pydantic w API (HTTP 422)
Nieistniejąca sesja	Złe API call	HTTPException 404
Nieprzewidziany błąd	Cokolwiek	Exception → logger.exception z pełnym stack trace

Wzorzec obsługi: Każdy typ błędu ma osobny handler except, kolejność od najbardziej szczegółowych do najogólniejszych. Po błędzie wiadomość użytkownika jest **cofana** z historii

(self.memory.history.pop()), żeby historia nie zawierała “wiszącej” wiadomości bez odpowiedzi. Zwracany jest ChatResult(ok=False, error=...) z czytelnym komunikatem.

Logowanie: - logs/chatbot.log – wszystkie zdarzenia (DEBUG+): inicjalizacja, każde zapytanie i odpowiedź (skrótowane do 120 znaków, dla bezpieczeństwa danych), liczba tokenów, długość historii. - logs/errors.log – tylko poziom ERROR – do szybkiego przeglądu problemów w produkcji. - Konsola – poziom INFO – do śledzenia w czasie rzeczywistym podczas developmentu.

Format logów zawiera datę, poziom, nazwę modułu i wiadomość, co pozwala łatwo filtrować (grep, narzędzia jak ELK / Loki).

8. Wdrożenie (ocena 5.0)

Sposób udostępnienia: Chatbot wystawiony jest jako usługa REST przez FastAPI (app/main.py).

Endpointy:

Metoda	Ścieżka	Opis
GET	/health	Health check (model + status)
POST	/sessions	Utwórz nową sesję, zwraca session_id (UUID4)
GET	/sessions	Lista aktywnych sesji
GET	/sessions/{id}	Informacje o sesji (długość historii, czasy)
DELETE	/sessions/{id}	Usuń sesję
GET	/sessions/{id}/history	Historia rozmowy
POST	/sessions/{id}/chat	Wyślij wiadomość, otrzymaj odpowiedź

Obsługa sesji użytkownika: - Klient tworzy sesję (POST /sessions) i otrzymuje session_id (UUID4). - Każda kolejna wiadomość zawiera ten identyfikator w URL. - SessionManager przechowuje słownik {session_id: Chatbot}. - Każda sesja ma **własną historię i własną**

instancję Chatbot – rozmowy są w pełni niezależne. - Sesje nieużywane przez godzinę są automatycznie kasowane (TTL), by zwolnić pamięć. - Dostęp do słownika sesji chroniony threading.Lock (thread-safe).

Uruchomienie lokalne:

```
ollama serve &  
ollama pull llama3.2  
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Dokumentacja interaktywna automatycznie pod `http://localhost:8000/docs`.

Jak system mógłby być wdrożony produkcyjnie:

1. **Konteneryzacja** – Dockerfile z aplikacją + `docker-compose.yml` łączące kontener aplikacji z kontenerem Ollamy. Każdy może zostać zskalowany niezależnie.
2. **Reverse proxy / TLS** – Nginx lub Traefik przed FastAPI, terminacja HTTPS, rate limiting, podstawowa autoryzacja.
3. **Skalowanie poziome** – wiele instancji aplikacji FastAPI za load balancerem. Ale: obecny SessionManager jest in-memory, więc trzeba by sesje wynieść do **Redis-a** (zmiana tylko warstwy SessionManager, reszta kodu bez zmian).
4. **Persystencja historii** – w obecnym kształcie historia żyje tylko w pamięci. W produkcji można zapisywać konwersacje w PostgreSQL / Mongo dla audytu i przywracania.
5. **Monitoring i metryki** – Prometheus endpoint, alerty na liczbę błędów, średni czas odpowiedzi, zużycie tokenów per sesja.
6. **Autoryzacja** – obecnie API jest otwarte. W produkcji: JWT / API keys, identyfikacja użytkownika (multi-tenant: jeden user = wiele sesji).
7. **Współbieżność** – FastAPI jest asynchroniczne (ASGI). Obecny kod używa synchronicznych endpointów, co dla LLM-ów (i tak wąskie gardło to model) jest OK. Dla wydajności można przejść na `async def` i `AsyncOpenAI`.
8. **Bezpieczeństwo promptów** – walidacja długości (już mamy w schemacie 8000 znaków), filtrowanie prompt injection, redaktowanie wrażliwych danych przed logowaniem.

9. **CI/CD** – testy jednostkowe (już można je dodać do Chatbot i ConversationMemory z fake klientem LLM), pipeline z lintem (ruff), buildem obrazu Docker i deploymentem.

Podsumowanie wymagań

Ocena	Wymaganie	Realizacja
3.5	Integracja z modelem	LLMClient przez Ollama (OpenAI-kompatybilne API)
3.5	Wysyłanie zapytań, generowanie odpowiedzi	Chatbot.chat() zwraca ChatResult
3.5	Prompt systemowy	W Settings.system_prompt, doklejany przy każdej generacji
3.5	Historia rozmowy	ConversationMemory.history jako lista {role, content}
3.5	Chatbot jako klasa	class Chatbot w app/chatbot.py
3.5	Parametry generacji	temperature=0.7, top_p=0.9, max_tokens=1024
4.0	Kontrola długości kontekstu	Sliding window po tokenach (tiktoken)
4.5	Obsługa błędów	8 typów wyjątków obsługiwanych odrębnie
4.5	Logowanie	chatbot.log (wszystko) + errors.log (tylko błędy)
5.0	REST API (FastAPI)	7 endpointów + Swagger pod /docs
5.0	Obsługa sesji	SessionManager z UUID4, TTL, thread-safe

Wszystkie wymagania na ocenę **5.0** zostały spełnione.